

**SQL MATCH_RECOGNIZE for Pandas
DataFrames:
A Python Implementation of SQL:2016
Row Pattern Recognition**

Master's Thesis
[University Name]

Submitted by
MonierAshraf

[Year]

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Objectives	1
1.3	Contributions	1
1.4	Organization of the Thesis	1
2	Background and Concepts	3
2.1	Pattern Matching in Sequential Data Analysis	3
2.1.1	Real-World Applications	4
2.2	The SQL:2016 MATCH_RECOGNIZE Clause	4
2.2.1	Clause Structure and Components	4
2.2.2	Illustrative Example: Stock Price V-Pattern	6
2.2.3	Comparison of Existing Implementations	7
2.3	Theoretical Foundations: Finite Automata	7
2.3.1	Non-deterministic Finite Automata (NFA)	8
2.3.2	Thompson’s Construction	8
2.3.3	Deterministic Finite Automata (DFA)	8
2.4	Compiler Design Principles	10
2.4.1	Lexical and Syntactic Analysis with ANTLR4	10
2.4.2	Abstract Syntax Tree (AST) Construction	11
2.4.3	Safe SQL Expression Evaluation	12
2.5	Pandas DataFrame Execution Model	13
2.6	Performance Optimization Techniques	14
2.6.1	LRU Pattern Caching	14
2.6.2	Memory Management	14
2.7	Related Work and Research Gap	14
2.7.1	Database System Implementations	14
2.7.2	Python Ecosystem Gap	15
2.7.3	Contribution of This Work	15
3	System Design and Architecture	16

4	Implementation	17
5	Experimental Evaluation	18
6	Conclusions and Future Work	19
	Bibliography	20

List of Figures

2.1	NFA for the pattern $A^+ B?$ illustrating ε -transitions and non-determinism	8
-----	--	---

List of Tables

2.1	Existing <code>MATCH_RECOGNIZE</code> implementations	7
2.2	Principal AST node types	12
2.3	Pattern cache configuration defaults	14

List of Algorithms

1	Thompson's NFA Construction	9
---	---------------------------------------	---

Chapter 1

Introduction

1.1 Problem Statement

[Write introduction here.]

1.2 Objectives

[State objectives here.]

1.3 Contributions

[List contributions here.]

1.4 Organization of the Thesis

The remainder of this thesis is organized as follows.

Chapter 2 – Background and Concepts presents the theoretical and technical foundations on which this work is built. It covers sequential pattern matching, the SQL:2016 `MATCH_RECOGNIZE` clause, finite automata theory (NFA, DFA, and their construction algorithms), compiler design principles (ANTLR4 parsing, AST construction, and safe expression evaluation), the Pandas DataFrame execution model, performance optimization techniques, and a survey of related systems.

Chapter 3 – System Design and Architecture describes the overall architecture of the proposed implementation, the component decomposition, and the design decisions that drove the system structure.

Chapter 4 – Implementation details the concrete realisation of each component: the grammar extension, the pattern tokenizer, the NFA/DFA engine, the match executor, the measure evaluator, and the production optimizations.

Chapter 5 – Experimental Evaluation presents the evaluation methodology, the benchmark datasets, and the experimental results covering correctness, performance scalability, pattern complexity, and comparison with existing systems.

Chapter 6 – Conclusions and Future Work summarizes the contributions, reflects on the limitations of the current implementation, and outlines directions for future research.

Chapter 2

Background and Concepts

This chapter establishes the theoretical and practical foundations required to understand the design and implementation of the `MATCH_RECOGNIZE` engine for Pandas DataFrames. Section 2.1 introduces the problem of sequential pattern detection in relational data. Section 2.2 examines the SQL:2016 `MATCH_RECOGNIZE` clause in detail. Section 2.3 covers the finite automata theory that underpins the pattern engine. Section 2.4 discusses the compiler design principles used for query parsing and safe evaluation. Section 2.5 describes the Pandas DataFrame execution model adopted by the implementation. Section 2.6 outlines the performance optimization techniques applied. Section 2.7 surveys related work and identifies the research gap addressed by this thesis.

2.1 Pattern Matching in Sequential Data Analysis

Sequential pattern detection represents a fundamental challenge in data analysis, where the objective is to identify ordered sequences of events or states within large datasets. Unlike traditional relational operations that focus on individual rows or aggregated values, pattern matching examines the temporal or logical ordering of data points to extract meaningful sequences. This capability is essential across many domains including financial market analysis, fraud detection, IoT sensor monitoring, and behavioural analytics.

Historically, relational database systems excelled at set-based operations—selection, projection, joins, and aggregation—but lacked expressive mechanisms for sequence analysis. Analysts requiring pattern detection were forced to either export data to specialised stream-processing systems, implement complex procedural logic using cursors and temporary tables, or employ application-layer processing with significant performance overhead. These limitations created a gap between the widespread availability of sequential data and the tools available to analyse it effectively. The introduction of row pattern recognition in SQL:2016 addresses this deficiency by

bringing sequence analysis capabilities directly into the query language [1].

2.1.1 Real-World Applications

Financial Market Analysis. Detecting V-shaped price patterns—price decline followed by recovery—requires examining ordered sequences of price movements. For example, identifying stocks whose price troughs are successively higher (a “higher-low” pattern) signals a potential upward trend. The canonical example used throughout this thesis is the detection of such V-patterns in customer order data [2].

Fraud Detection. Credit card fraud often manifests as specific transaction sequences, such as multiple small test charges followed by a large purchase, or geographically impossible transactions occurring within unrealistic timeframes. Expressing these rules declaratively in SQL is more maintainable than equivalent procedural logic.

IoT and Sensor Data. Manufacturing systems monitor sensor readings to detect equipment degradation patterns, such as a gradual temperature rise followed by a sudden drop, which may indicate imminent failure. Pattern queries over time-series sensor streams are a natural fit for `MATCH_RECOGNIZE`.

Clickstream Analysis. E-commerce platforms analyse user navigation sequences to identify conversion paths or abandonment sequences, for example “view → add-to-cart → remove → exit,” which indicates purchase hesitation.

2.2 The SQL:2016 `MATCH_RECOGNIZE` Clause

The `MATCH_RECOGNIZE` clause was introduced in ISO/IEC 9075-2:2016 (SQL:2016, Part 2: Foundation) as a declarative, SQL-native mechanism for identifying patterns in ordered sequences of rows [1]. It brings capabilities previously available only in specialised stream-processing systems into standard relational databases.

2.2.1 Clause Structure and Components

The clause operates on a rowset (table or derived table) and consists of seven interconnected sub-clauses, as shown in Listing 2.1.

```
1 SELECT <columns>  
2 FROM <table>  
3 MATCH_RECOGNIZE (
```

```

4  PARTITION BY <partition_columns>    -- 1. Data
      organisation
5  ORDER BY      <ordering_columns>    -- 2. Row sequencing
6  MEASURES      <expressions>         -- 3. Output
      computations
7  <rows_per_match>                      -- 4. Result
      granularity
8  AFTER MATCH SKIP <option>           -- 5. Overlap handling
9  PATTERN ( <pattern_regex> )         -- 6. Sequence
      specification
10 DEFINE <variable_definitions>      -- 7. Variable
      predicates
11 ) AS <alias>

```

Listing 2.1: General syntax of the `MATCH_RECOGNIZE` clause

1. **PARTITION BY.** Divides the input rowset into independent partitions, analogous to window-function partitioning. Pattern matching runs independently within each partition, enabling both parallel processing and domain-specific grouping (e.g. per customer or per sensor).
2. **ORDER BY.** Establishes the logical ordering of rows within each partition. The pattern-matching algorithm processes rows strictly according to this ordering. Multiple sort keys with `ASC/DESC` modifiers are supported.
3. **MEASURES.** Defines computed expressions that extract values from matched row sequences using pattern variables and navigation functions (`FIRST`, `LAST`, `PREV`, `NEXT`). Each `MEASURE` expression becomes a column in the output.
4. **Output options.**
 - `ONE ROW PER MATCH`: one summary row per complete match.
 - `ALL ROWS PER MATCH`: one row for every row participating in a match.
 - `ALL ROWS PER MATCH WITH UNMATCHED ROWS`: includes non-matching rows in the result.
5. **AFTER MATCH SKIP.** Controls how the engine resumes after finding a match:
 - `SKIP TO NEXT ROW` – resume from the row after match start (maximum overlaps).
 - `SKIP PAST LAST ROW` – resume from the row after match end (non-overlapping matches).

- SKIP TO FIRST *var* – resume from the first row classified as *var*.
 - SKIP TO LAST *var* – resume from the last row classified as *var*.
6. **PATTERN.** Specifies the sequence using regular-expression-like syntax with pattern variables as terminals:
- Concatenation: A B C (A followed by B followed by C).
 - Alternation: A | B (either A or B).
 - Quantifiers: A* (zero or more), A+ (one or more), A? (zero or one), A{n} (exactly *n*), A{n,m} (between *n* and *m*).
 - Grouping: (A B)+ (one or more AB pairs).
 - Anchors: ^A B C\$ (pattern must span the entire partition).
 - Exclusions: {- A+ B+ -} (rows *not* matching the enclosed sub-pattern).
 - Permutations: PERMUTE(A, B, C) (any ordering of A, B, and C).
7. **DEFINE.** Assigns Boolean predicates to pattern variables. Each row is evaluated against these predicates to determine its variable classification. Predicates may reference current-row columns, navigation functions, aggregations over matched sub-sequences, and Boolean/comparison operators.

2.2.2 Illustrative Example: Stock Price V-Pattern

Listing 2.2 shows a complete MATCH_RECOGNIZE query that detects V-shaped price patterns in customer order data.

```

1 SELECT customer_id,
2         start_price, bottom_price, final_price,
3         start_date,  final_date
4 FROM   orders
5 MATCH_RECOGNIZE (
6     PARTITION BY customer_id
7     ORDER BY     order_date
8     MEASURES
9         START.price           AS start_price,
10        LAST(DOWN.price)      AS bottom_price,
11        LAST(UP.price)        AS final_price,
12        START.order_date      AS start_date,
13        LAST(UP.order_date)   AS final_date
14 ONE ROW PER MATCH
15 AFTER MATCH SKIP PAST LAST ROW

```

```

16  PATTERN (START DOWN+ UP+)
17  DEFINE
18      DOWN AS price < PREV(price),
19      UP   AS price > PREV(price)
20 );

```

Listing 2.2: Detecting V-shaped price patterns with `MATCH_RECOGNIZE`

The engine partitions rows by `customer_id`, sorts by `order_date`, and seeks sequences matching `START DOWN+ UP+`: one `START` row (any row, as it carries no `DEFINE` predicate), one or more `DOWN` rows (each with a strictly lower price than its predecessor), and one or more `UP` rows (each with a strictly higher price than its predecessor). For every match found, the `MEASURES` sub-clause extracts the starting price and date from the `START` row, the lowest price from the last `DOWN` row, and the final price from the last `UP` row.

2.2.3 Comparison of Existing Implementations

Several database and stream-processing systems have implemented `MATCH_RECOGNIZE`; Table 2.1 compares the most prominent ones.

Table 2.1: Existing `MATCH_RECOGNIZE` implementations

System	Compliance	Deployment	Licence	Characteristics
Oracle DB	Full SQL:2016	On-premise / Cloud	Commercial	First implementation (12c, 2013)
Trino	Extensive	Distributed cluster	Apache 2.0	Cloud-native, federated queries
Apache Flink	CEP library	Stream processor	Apache 2.0	Real-time; separate CEP API
IBM Db2	Partial	On-premise / Cloud	Commercial	Mainframe focus
Databricks	SQL Analytics	Cloud SaaS	Commercial	Spark-based lakehouse
This work	Full SQL:2016	In-process Python	Open-source	Zero-deployment; Pandas-native

The key limitation shared by all existing solutions is that none integrates natively with Python’s Pandas ecosystem, which is the dominant platform for in-process data science workloads. This thesis addresses that gap.

2.3 Theoretical Foundations: Finite Automata

Pattern matching in `MATCH_RECOGNIZE` is fundamentally a problem of sequence recognition. A sequence of data rows, each labelled with the pattern variable(s) it satisfies, constitutes a string over the alphabet of pattern variables. The problem of determining whether this string is accepted by the pattern is exactly the problem solved by finite automata [3].

2.3.1 Non-deterministic Finite Automata (NFA)

A *Non-deterministic Finite Automaton* (NFA) is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of *states*;
- Σ is the *input alphabet* (pattern variables in our setting);
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$ is the *transition function*, mapping each state and symbol (or the empty string ε) to a *set* of successor states;
- $q_0 \in Q$ is the *initial state*; and
- $F \subseteq Q$ is the set of *accepting states*.

NFAs naturally model alternation and optional constructs owing to their ability to be in multiple states simultaneously and to use ε -transitions (transitions that consume no input).

Figure 2.1 shows the NFA for pattern $A^+ B?$.

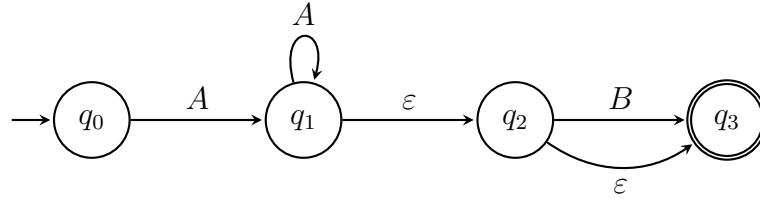


Figure 2.1: NFA for the pattern $A^+ B?$ illustrating ε -transitions and non-determinism

2.3.2 Thompson's Construction

Thompson's construction algorithm [4] converts a regular expression into an NFA in linear time $O(n)$, where n is the length of the expression. The algorithm proceeds recursively over the expression structure according to the rules in Algorithm 1.

The resulting NFA has at most $2n$ states and is constructed in $O(n)$ time and space, making it suitable for patterns of arbitrary complexity [5].

2.3.3 Deterministic Finite Automata (DFA)

A *Deterministic Finite Automaton* (DFA) is a special case of an NFA in which: (i) there are no ε -transitions, and (ii) for each state q and symbol a , $|\delta(q, a)| = 1$ (exactly one successor state). DFAs require $O(n)$ time to process an input of length n with no backtracking, making them ideal for row-by-row matching.

Algorithm 1 Thompson's NFA Construction

```
1: procedure BUILD( $e$ )
2:   if  $e = \varepsilon$  then
3:     return NFA with one  $\varepsilon$ -transition from start to accept
4:   else if  $e = a \in \Sigma$  then
5:     return NFA with one  $a$ -labelled transition from start to accept
6:   else if  $e = r \cdot s$  (concatenation) then
7:      $N_r \leftarrow \text{BUILD}(r)$ ;  $N_s \leftarrow \text{BUILD}(s)$ 
8:     Merge accept state of  $N_r$  with start state of  $N_s$ 
9:     return combined NFA
10:  else if  $e = r \mid s$  (alternation) then
11:     $N_r \leftarrow \text{BUILD}(r)$ ;  $N_s \leftarrow \text{BUILD}(s)$ 
12:    Add new start  $q_{\text{new}}$  with  $\varepsilon$ -transitions to  $q_0^{N_r}$  and  $q_0^{N_s}$ 
13:    Add new accept  $f_{\text{new}}$  with  $\varepsilon$ -transitions from  $F^{N_r}$  and  $F^{N_s}$ 
14:    return combined NFA
15:  else if  $e = r^*$  (Kleene star) then
16:     $N_r \leftarrow \text{BUILD}(r)$ 
17:    Add  $\varepsilon$ -transition from start to accept (zero repetitions)
18:    Add  $\varepsilon$ -transition from  $F^{N_r}$  back to  $q_0^{N_r}$  (repetition)
19:    return modified NFA
20:  else if  $e = r^{\{n,m\}}$  (bounded quantifier) then
21:    Construct  $n$  mandatory copies followed by  $(m - n)$  optional copies
22:    return combined NFA
23:  end if
24: end procedure
```

Subset Construction (NFA $\rightarrow$ DFA). The *subset construction* (powerset construction) algorithm converts an NFA to an equivalent DFA. Each DFA state corresponds to a *set* of NFA states. For a DFA state S and input symbol a :

$$\delta_{\text{DFA}}(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta_{\text{NFA}}(q, a)\right)$$

where $\varepsilon\text{-closure}(T)$ is the set of all NFA states reachable from any state in T by zero or more ε -transitions. Although the resulting DFA may have up to $2^{|Q|}$ states in the worst case, patterns arising from `MATCH_RECOGNIZE` syntax are structurally constrained and in practice yield DFAs with far fewer states.

DFA Minimisation. Hopcroft’s algorithm [3] reduces a DFA to its canonical minimal form in $O(n \log n)$ time by iteratively partitioning states into equivalence classes of indistinguishable states. The implementation applies minimisation after subset construction to reduce memory footprint and improve matching speed.

State Prioritisation for SQL Semantics. SQL:2016 mandates *greedy* matching by default: the engine must prefer the longest possible match, and in the case of alternation, the leftmost alternative takes precedence. Standard DFA theory does not encode these preferences. The implementation addresses this by:

1. Assigning a *priority* to each NFA transition based on its position in the pattern and quantifier greediness.
2. Propagating priorities through subset construction so that each DFA transition inherits the minimum (highest-priority) value of the contributing NFA transitions.
3. Ordering outgoing DFA transitions by priority during match execution so that higher-priority paths are explored first.

2.4 Compiler Design Principles

The implementation applies classical compiler construction techniques to transform SQL text into executable pattern-matching logic in a pipeline of four stages: lexical analysis, syntactic analysis, AST construction, and safe code evaluation.

2.4.1 Lexical and Syntactic Analysis with ANTLR4

ANTLR4 (ANother Tool for Language Recognition, version 4) is a parser generator that produces top-down, recursive-descent parsers from context-free grammar speci-

fications written in the ANTLR meta-language [6]. Given a grammar file the tool generates a lexer, a parser, and listener/visitor scaffolding in the target language (Python in this project).

The implementation extends Trino’s production SQL grammar (`TrinoLexer.g4` / `TrinoParser.g4`) with specialised rules for every sub-clause of `MATCH_RECOGNIZE`, as shown in the excerpt in Listing 2.3. A custom error listener intercepts syntax errors and reports them with precise line/column positions and contextual suggestions.

```

1 matchRecognize
2     : MATCH_RECOGNIZE '('
3         partitionBy? orderBy? measures?
4         rowsPerMatch afterMatchSkip?
5         pattern subsetClause? define
6         ')' (AS? identifier)?
7     ;
8
9 rowPattern
10    : rowPatternTerm
11    | rowPattern '|' rowPatternTerm
12    ;
13
14 rowPatternFactor
15    : rowPatternPrimary rowPatternQuantifier?
16    ;
17
18 rowPatternPrimary
19    : identifier                                # patternVariable
20    | '^'                                       # anchorStart
21    | '$'                                       # anchorEnd
22    | '(' rowPattern ')'                       # patternGroup
23    | '{-' rowPattern '-}'                     # patternExclusion
24    | PERMUTE '(' rowPattern
25        (',' rowPattern)* ')'                 # patternPermute
26    ;

```

Listing 2.3: ANTLR4 grammar excerpt for `MATCH_RECOGNIZE`

2.4.2 Abstract Syntax Tree (AST) Construction

Parse trees produced by ANTLR4 contain many structural nodes (keywords, punctuation) that are unnecessary for execution. A visitor traverses the parse tree and produces a compact *Abstract Syntax Tree* (AST) whose node types correspond directly to semantic constructs [5]. The key node types are summarised in Table 2.2.

The AST is validated for semantic correctness before pattern compilation: un-

Table 2.2: Principal AST node types

Node Type	Represents
MatchRecognizeNode	Root of the clause
SequenceNode	Concatenation of sub-patterns
AlternationNode	Alternation (A B)
QuantifiedNode	Quantified pattern (A+, B{n,m})
PermuteNode	PERMUTE(A, B, C) construct
ExclusionNode	{- pattern -} exclusion
AnchorNode	^ / \$ anchor
MeasureSpec	Single MEASURES expression
ConditionSpec	Single DEFINE predicate

defined variable references in DEFINE predicates, circular definitions, and illegal quantifier ranges are all detected and reported at this stage.

2.4.3 Safe SQL Expression Evaluation

DEFINE predicates and MEASURES expressions are user-supplied strings that must be evaluated as Python code at runtime. Directly calling `eval()` on arbitrary user input creates a code-injection vulnerability. The implementation instead uses Python’s `ast` module to parse and validate expressions before execution, as shown in Listing 2.4.

```

1  import ast
2
3  def safe_evaluate(sql_expr, row_data, context):
4      # 1. Translate SQL syntax to Python equivalents
5      py_expr = translate_sql_to_python(sql_expr)
6
7      # 2. Parse into a Python AST (expression mode only)
8      tree = ast.parse(py_expr, mode='eval')
9
10     # 3. Whitelist-validate: allow comparisons, arithmetic,
11     #    boolean ops, literals, subscripts -- forbid imports,
12     #    function calls (except navigation functions),
13     #    assignments
14     ASTSecurityValidator().visit(tree)
15
16     # 4. Compile and execute in a controlled namespace
17     code = compile(tree, '<expr>', 'eval')
18     return eval(code, {'__builtins__': {}},
19                   {'row': row_data, **context})

```

Listing 2.4: Safe SQL expression evaluation using AST whitelisting

The controlled namespace exposes only the current row dictionary and the whitelisted navigation functions (`PREV`, `NEXT`, `FIRST`, `LAST`), providing a safe sandbox with no access to the file system, network, or interpreter internals.

2.5 Pandas DataFrame Execution Model

Pandas [7] is a Python library providing the `DataFrame` data structure: a two-dimensional, labelled table backed by typed NumPy arrays. Its in-memory, columnar storage enables vectorised operations over millions of rows.

The `MATCH_RECOGNIZE` engine adopts the following execution strategy over a `DataFrame D`:

1. **Partition.** Group D by the `PARTITION BY` columns using `DataFrame.groupby()`, producing a collection of independent sub-frames.
2. **Order.** Sort each partition by the `ORDER BY` columns using `sort_values()`.
3. **Classify rows.** For each row, evaluate every `DEFINE` predicate and record which pattern variables the row satisfies.
4. **Simulate the DFA.** Feed the classified sequence into the compiled DFA row by row. On reaching an accepting state, record the matched row indices and the active variable assignments.
5. **Evaluate measures.** For each recorded match, evaluate the `MEASURES` expressions using the variable assignments and navigation functions.
6. **Assemble output.** Concatenate match results into the output `DataFrame` with the declared measure columns.

Navigation functions. During measure evaluation, navigation functions resolve to direct `DataFrame` row accesses:

- `FIRST(var.col) → df.iloc[match[var][0]]['col']`
- `LAST(var.col) → df.iloc[match[var][-1]]['col']`
- `PREV(col, k) → df.iloc[current_idx - k]['col']`
- `NEXT(col, k) → df.iloc[current_idx + k]['col']`

2.6 Performance Optimization Techniques

2.6.1 LRU Pattern Caching

Pattern compilation—tokenisation $\rightarrow$ NFA construction $\rightarrow$ DFA conversion $\rightarrow$ minimisation—is computationally expensive, especially for patterns with quantifiers and PERMUTE constructs. The implementation maintains a thread-safe *Least Recently Used* (LRU) cache that maps a hash of the pattern string and DEFINE predicates to the compiled DFA. Key configuration parameters (configurable via environment variables) are listed in Table 2.3.

Table 2.3: Pattern cache configuration defaults

Parameter	Default	Description
CACHE_SIZE_LIMIT	10 000 entries	Maximum cached patterns
CACHE_MEMORY_LIMIT_MB	500 MB	Maximum cache memory
CACHE_TTL_SECONDS	3 600 s	Entry time-to-live
CACHE_CLEAR_THRESHOLD_MB	400 MB	Proactive eviction threshold

2.6.2 Memory Management

Large-scale matching over millions of rows requires careful memory management. The implementation processes one partition at a time, accumulating matches in a fixed-size buffer before flushing results to the output DataFrame. Memory usage is monitored via `psutil`; garbage collection is triggered proactively when consumption approaches user-configured limits. Object pools reuse DFA state structures and transition objects across successive matches, reducing allocator pressure.

2.7 Related Work and Research Gap

2.7.1 Database System Implementations

Oracle Database 12c [2] introduced the first mainstream implementation of MATCH_RECOGNIZE in 2013 with full SQL:2016 compliance, cost-based optimisation plans, and support for parallel partition processing. Its limitations include high enterprise licensing costs and mandatory deployment of the full Oracle RDBMS stack.

Apache Flink CEP [8] provides low-latency pattern detection on streaming data with exactly-once processing semantics. However, its pattern API is separate from SQL and requires learning a distinct programming model. Furthermore, Flink

is designed for continuous streaming workloads, adding unnecessary complexity for batch analytics.

Trino [9] (formerly PrestoSQL) is an open-source distributed query engine with extensive `MATCH_RECOGNIZE` support. It is designed for interactive, federated queries over cloud data sources. Deploying even a single-node Trino instance requires Java, significant configuration, and acts as an external service rather than an embedded library.

2.7.2 Python Ecosystem Gap

Despite Python’s dominance in data science, no maintained library implements SQL:2016 `MATCH_RECOGNIZE` semantics for Pandas DataFrames. Existing approaches require analysts to manually implement stateful logic using `DataFrame.apply()` chains, `iterrows()`, or external database round-trips—all of which are error-prone, verbose, and difficult to maintain. Alternative DataFrame libraries—Dask, Polars, Modin—similarly lack sequential pattern primitives.

2.7.3 Contribution of This Work

This thesis addresses the identified gap with the following contributions:

1. The **first SQL:2016-compliant `MATCH_RECOGNIZE` implementation** that runs entirely in-process within a Python Pandas workflow with zero additional deployment.
2. **Complete pattern construct support**, including quantifiers, alternation, grouping, anchors, exclusions, and `PERMUTE`, faithfully following the SQL:2016 standard.
3. A **production-ready optimisation layer** comprising LRU pattern caching, partition-wise memory management, and thread-safe concurrent execution.
4. A **safe expression evaluation pipeline** that prevents code-injection vulnerabilities without sacrificing the full expressiveness of user-defined predicates.
5. **Empirical validation** on a real-world dataset of over 2.2 million rows with 100% correctness across 25 diverse test patterns and throughput reaching up to 12918 rows/sec on a commodity laptop.

Chapter 3

System Design and Architecture

[Write system design here.]

Chapter 4

Implementation

[Write implementation here.]

Chapter 5

Experimental Evaluation

[Write experimental evaluation here.]

Chapter 6

Conclusions and Future Work

[Write conclusions here.]

Bibliography

- [1] ISO/IEC. *ISO/IEC 9075-2:2016 Information technology — Database languages — SQL — Part 2: Foundation (SQL/Foundation)*. International Standard. International Organization for Standardization, 2016.
- [2] Oracle Corporation. *Pattern Matching in Oracle Database 12c*. https://docs.oracle.com/database/121/DWHSG/pattern_match.htm. Accessed: 2024. 2013.
- [3] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-45536-9.
- [4] Ken Thompson. “Programming Techniques: Regular Expression Search Algorithm”. In: *Communications of the ACM* 11.6 (1968), pp. 419–422. DOI: 10.1145/363347.363387.
- [5] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd. Boston, MA, USA: Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.
- [6] Terence Parr. *The Definitive ANTLR 4 Reference*. Lewisville, TX, USA: Pragmatic Bookshelf, 2013. ISBN: 978-1-934356-99-9.
- [7] Wes McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a.
- [8] Alexander Alexandrov et al. “The Stratosphere Platform for Big Data Analytics”. In: *The VLDB Journal*. See also: Apache Flink CEP documentation <https://nightlies.apache.org/flink/flink-docs-stable/docs/libs/cep/>. 2014.
- [9] Trino Project. *Trino Documentation: Pattern Recognition in Window Functions*. <https://trino.io/docs/current/sql/match-recognize.html>. Accessed: 2024. 2023.